

AST326Y1 Group H

Lab 1 - Basic Photon Statistics
Week 2 Mini-Report

Plan

- 4.1: Data distributions
- 4.2: Discovering the CMOS! (4.2.1-4.2.3)
- 4.2.4: Read Noise
 - Recording Data
 - Data Processing
- The +
- The -
- Further Investigations for 4.2.5
- What do we have to say for other groups?

4.1: Data Distributions

Recreating the Poisson distribution histogram (figure 7)

- **Hardcoded** data list
- Poisson pmf **amplified** to be visible
- Using **pyplot.Axes** for greater range of plot modifications (ticks etc.)

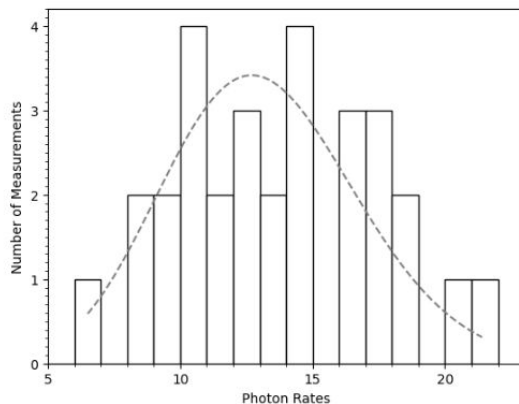
```
1 import numpy as np
2 from scipy.special import factorial
3 import matplotlib.pyplot as plt
```

Part 1: Table sample set

This sample set needs to be hardcoded since there is no file to read from. The goal is to recreate a figure similar to *Figure 7* in the handout.

```
In [3]: 1 """
2 Part 1: Small sample set
3 This part uses data from the table in section 4.1 of the lab handout to
4 recreate figure 7.
5 """
6 p1_spl = [13, 17, 18, 14, 11, 8, 21, 18, 9, 12,
7           9, 17, 14, 6, 10, 16, 16, 11, 10, 12,
8           8, 20, 14, 10, 14, 17, 13, 16, 12, 10]
9
10 p1_stddev = np.std(p1_spl)
11 p1_mean = np.mean(p1_spl)
12
13 ax = plt.axes()
14 plt.hist(p1_spl, bins=17, range=(5,22), edgecolor="black", color="white")
15 plt.xlim(5, 23)
16
17 ax.set_yticks(np.arange(0, 5, 1))
18 ax.set_xticks(np.arange(5, 25, 5))
19
20 ax.set_yticks(np.arange(0, 4.6, 0.1), minor=True)
21 ax.set_xticks(np.arange(5, 23, 1), minor=True)
22
23 plt.xlabel('Photon Rates')
24 plt.ylabel('Number of Measurements')
25
26 # Poisson distribution overplot
27 x = np.arange(6.5, 21.5, 0.1)
28 y = np.exp(-p1_mean) * np.power(p1_mean, x)/factorial(x)
29
30 amp = 31 # amplify the presence of the Poisson distribution
31 ax.plot(x, y * amp, color="gray", linestyle="dashed")
```

Out[3]: [matplotlib.lines.Line2D at 0x7fef53863d70]

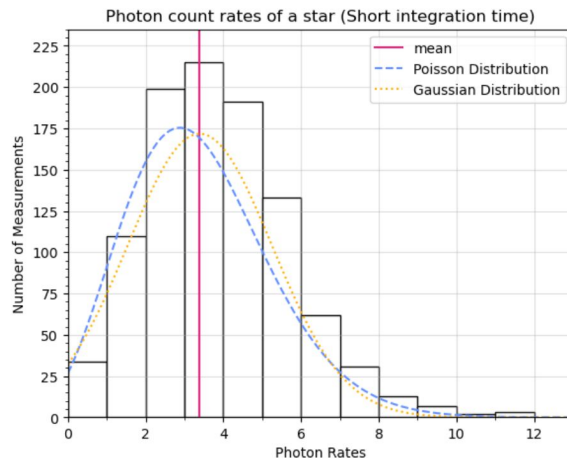


4.1: Data Distributions

Small.txt, Large.txt

- Reading from files using `np.loadtxt()`, σ and μ computed with NumPy methods
- **Issue:** factorial overflows in Poisson pmf $\rightarrow$ loop division

```
1 def pois(x_arr, mu):
2     """
3     Return Poisson distribution using integer array x_arr and mean mu.
4     Used to counteract scipy.special.factorial() overflow for p2_large.
5     """
6     result = np.exp(-mu, dtype=np.float128) \
7         * np.power(mu, x_arr, dtype=np.float128)
8
9     # division by factorial, implemented by integer
10    for i in range(0, len(result) - 1):
11        for j in range(1, x_arr[i] + 1):
12            result[i] /= j
13    return result
```



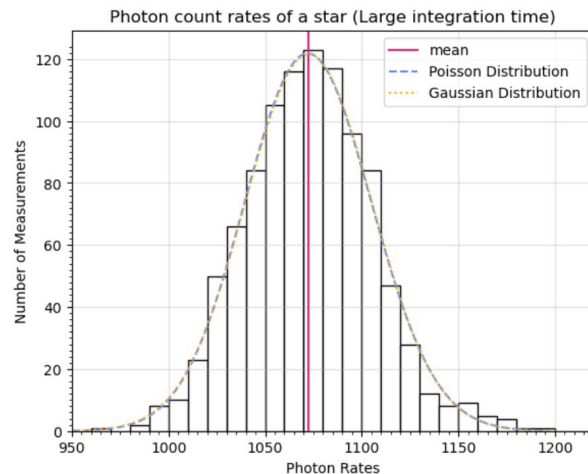
small.txt photon
count rates

Small $\sim$ Poisson
($\mu \approx \sigma^2$)

Small mean: 3.392 , Small StDev: 1.8580462857528603
Large mean: 1072.528 , Large StDev: 32.8154722044343

large.txt photon
count rates

Large $\sim$ Normal(?)
Poisson $\rightarrow$ Normal
with $\mu \rightarrow \infty$



4.2: Discovering the CMOS! (4.2.1-4.2.3)

We had the monochrome CMOS
FMVU-03MTM

- First practical
 - No stand
 - Problem with the focus
 - No photos taken
- Second practical
 - We had a stand
 - Clear focused image
 - Photos taken

4.2.1

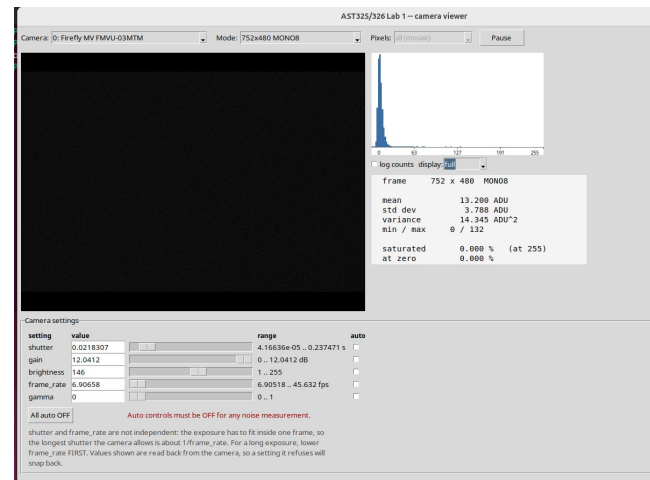
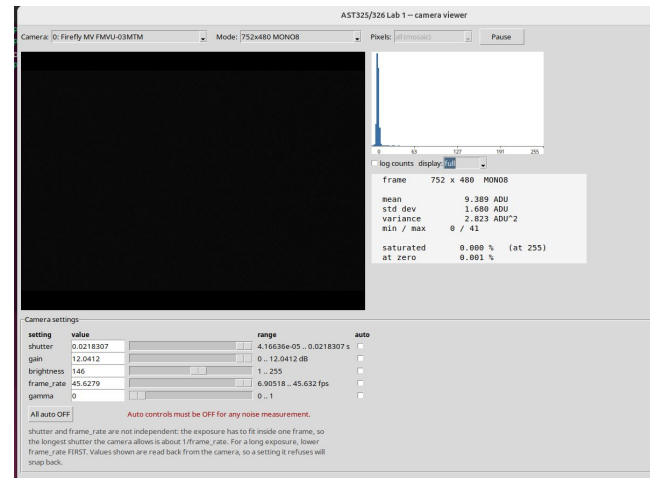
- lowest shutter - mean 53.9 (sd 7.7), one peak
- increase shutter = brighter values, more spread out, multiple peaks.
 - Some pixels get saturated at high (0.02s) shutter speed pointed at blend of desk and chair
- max value for an 8-bit camera = 255

4.2: Discovering the CMOS! (4.2.1-4.2.3)

4.2.2 - In the Dark!

Lowest Frame rate -> Longest exposure

- min frame 6.90518 - couple brighter bins but still really low - max shutter 0.27
- A couple of individually bright pixels
- Dark and uniform but not static!
 - Exposure time increase
 - increasing rate makes it darker
 - the individually dark pixels darkened/disappeared
 - max 45.632 - max shutter 0.02

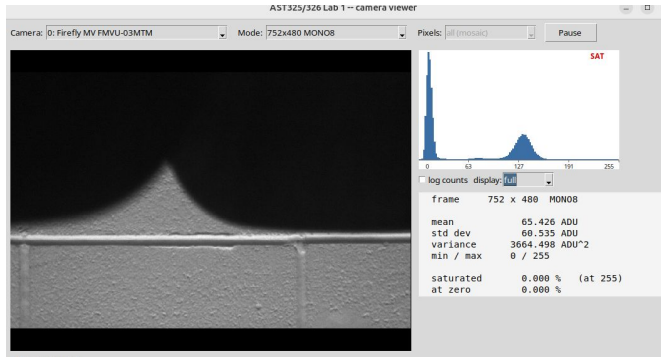


4.2.3 - Uniform vs Non-uniform

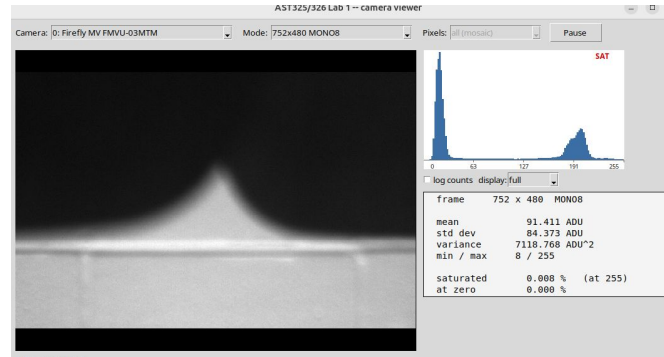
Used the back of a chair

Set max gain, aperture F/1.4, mean ADU set 180-220

- In focus we saw stark peaks, while out of focus smoothed out the histogram
- We also saw an increase in saturation - counterintuitive because less lens area should give us less light.
- Once fully closed, it was dark (obviously)



Lowering
Aperture
→



(data from first week)

F/1.4: mean 221

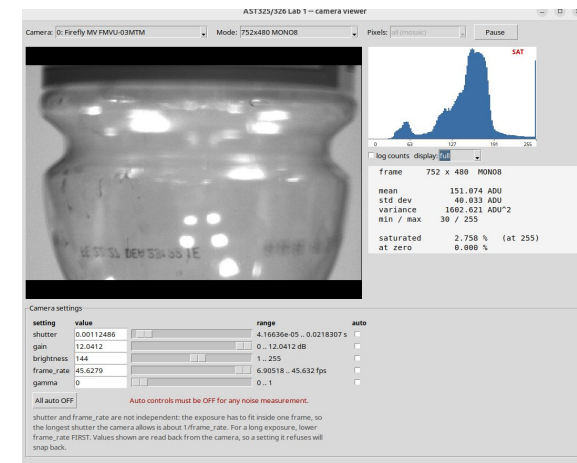
STD: 24.4

F/C: mean 54.5

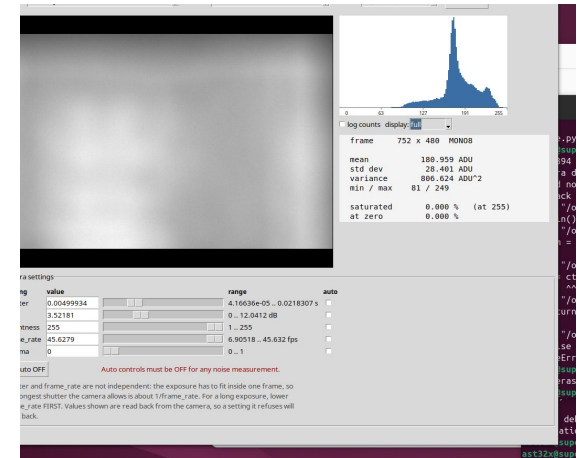
STD: 8.1

4.2.4: Read Noise

- We started with a water bottle, for a non-uniform object, but it became a problem with how reflective it was, our histogram was changing too much. Anyone moving in the room changed the reflection!
- Switched to a stable, less reflective object for more consistent captures.
- We also blurred the camera so we got closer to a gradient
- We then were able to take light and dark captures to use in the following coding sections



Reflective object (not blurred)



Improved non-reflective object (blurred)

4.2.4: Read Noise

1,000 random samples from 360,960 pixels.

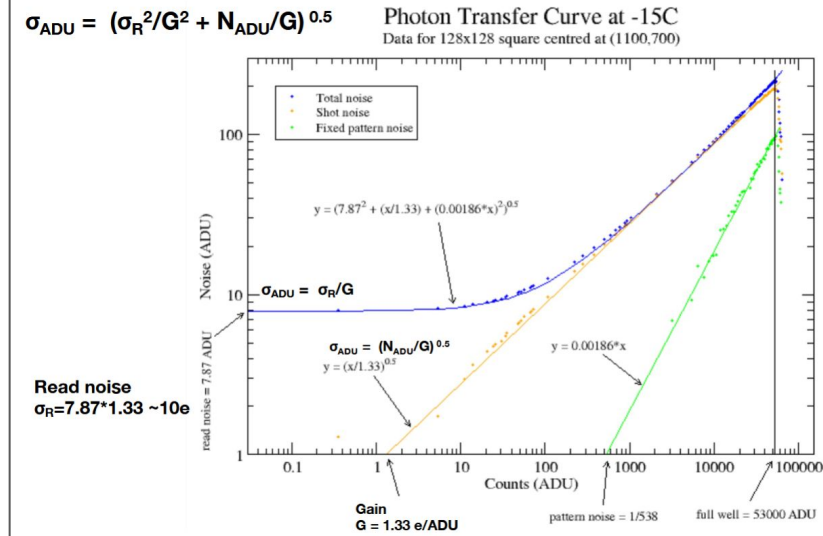
In-house **least-squares** method

```
def least_squares(x_arr, y_arr, N):  
    """  
    Return a tuple of gradient (m) and constant (c) for linear regression using values from arrays x_arr and y_arr.  
    Both arrays contain N entries.  
    """  
    x_arr2 = x_arr ** 2  
    xy_arr = x_arr * y_arr  
  
    sum_x = np.sum(x_arr)  
    sum_y = np.sum(y_arr)  
    sum_x2 = np.sum(x_arr2)  
    sum_xy = np.sum(xy_arr)  
  
    det = (N * sum_x2 - sum_x ** 2)**(-1)  
  
    m = det * (sum_xy * N - sum_y * sum_x)  
    c = det * (sum_y * sum_x2 - sum_xy * sum_x)  
  
    return (m, c)
```

4.2.4: Read Noise

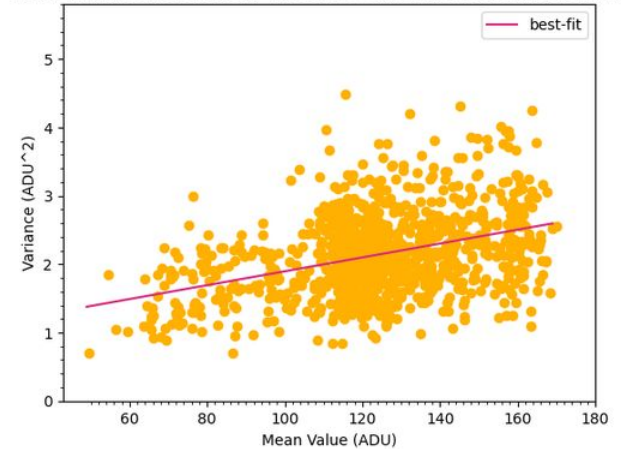
Questions/Concerns:

- Axis **powers**? Which value on which axis?
 - Tried to take reference on lecture **slides**:
Gain is **reciprocal** of slope, units **e⁻/ADU**
(**Logarithmic** axes?)



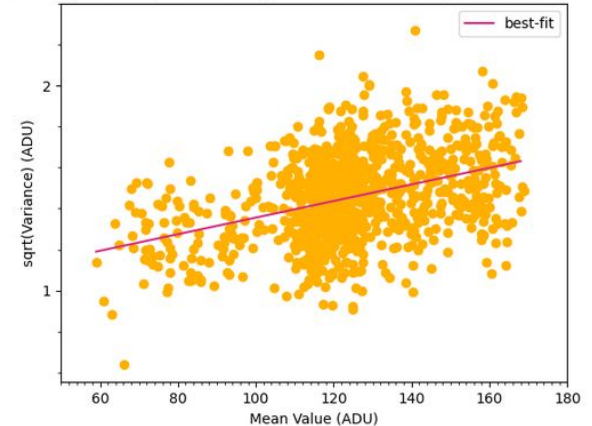
Gain: 98.55761779793727 e/ADU; Noise: 0.8808737963019396 ADU

Mean-Variance Plot of Firefly MV CMOS Camera (1000 random samples)



Gain: 246.85517678277273 e/ADU; Noise: 0.950749733476711 ADU

Mean-sqrt(Variance) Plot of Firefly MV CMOS Camera (1000 random samples)



4.2.4: Read Noise

Piazza post response!

(28/09/2026)

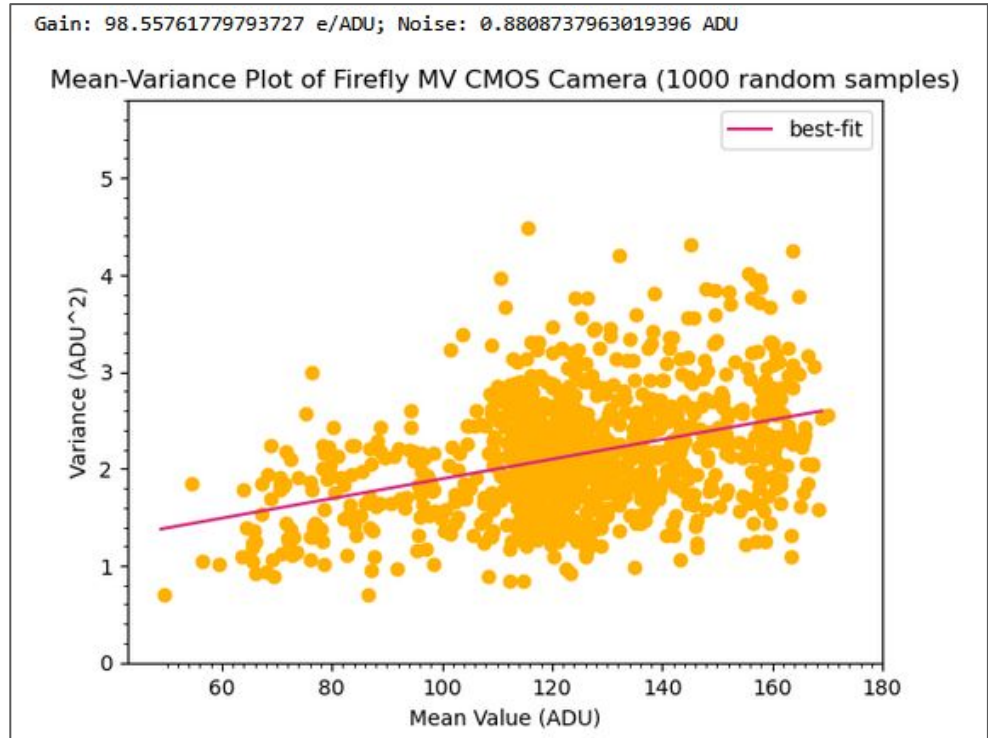
Plot σ^2 against μ , no changes in power.

→ **Top plot** was the correct one!

- **Gain:** 98.6 e^-/ADU
- **Read Noise:** 0.881 ADU
(86.8 e^-)

Thank you Bruno!

In hindsight, dimensional analysis would have been a good way to find out



The +

- The second week, we knew what to do hence:
 - Smoother Workflow
 - We were more efficient and productive
 - We obtained better results
 - Better teamwork
 - We understood each other's strength better
 - Result matches Theory
 - Small data set approached a Poisson distribution as expected
 - Large data set approached a Normal distribution as expected
 - We were more confident about our results
- We set up a GitHub repo (thank you Group D!)

The -

- First time with the camera resulted in inconclusive results (due to it not being able to be focused)
- We did not take any pictures from the 1st hour - quickly went back to what we did in the 2nd hour to take screenshots
 - Both caused lots of wasted time, but was made up in our efficiency in the second hour
- At first we couldn't figure out the capturing process
- Choosing a good object for the camera was a challenge, originally using one with lots of reflections was a mistake.
- Linux terminal was hard to get used to

Further Investigations (4.2.5) Measuring Dark Current

The slope of the best-fit line will output ADU/(seconds*pixels). To convert it to electrons per second per pixel we have to use the formula:

$$D(\text{electrons per pixel per second}) = D(\text{ADU}) * CF * (\text{electrons per ADU})$$

where CF is the gain we measured at 4.2.4 which we will use to convert dark current into electrons per second per pixel.

Normally there is a linear relation between electrons = dark current rate * time. But as exposure time increases and image comes closer to saturation each pixel's potential well gets piled up with electrons. Also gets limited by ADU's max values which is $2^n - 1$, 255 (for our camera), therefore reported value of pixels mean plateaus which leaves the linear regime. Due to these upper bounds for ADU we will be setting the exposure time in milliseconds and up to 30ms max to keep electrons = dark current rate * time linear.

What do we have to say for other groups?

- If in doubt: Turn off and on
 - If in more doubt, ask a TA/CI/T not A
- Take lots of pictures and notes
- Make a GitHub repo! It's a learning curve but very worth it.
- Take more frames than you think you need. More data makes the distributions much cleaner, especially for checking Poisson against Normal.

QUESTIONS / COMMENTS ?